

Collectiv

Sistema di Catalogazione per Collezionisti

Progetto di TPSIT

Classe 5CII

Autori Girolimetto Riccardo
Muca Alesander
Touel Adem

Data 13 Maggio 2026

Stack PHP + MySQL + Flutter

PHP

MySQL

Flutter

REST API

Token Bearer

Indice

1	Panoramica Generale	3
2	Architettura del Sistema	3
2.1	Flusso di una richiesta tipica	3
2.2	Struttura delle directory	4
3	Database	4
3.1	Tabella <code>users</code>	5
3.2	Tabella <code>categories</code>	5
3.3	Tabella <code>items</code>	5
3.4	Schema Relazioni (E/R)	6
4	Backend (PHP)	6
4.1	File di Configurazione	6
4.2	Router Principale (<code>index.php</code>)	7
4.3	Moduli di Supporto (<code>core/</code>)	7
4.3.1	<code>core/Auth.php</code>	7
4.3.2	<code>core/Response.php</code>	7
4.4	Controller API	7
4.4.1	<code>api/Auth.php</code> — Autenticazione	7
4.4.2	<code>api/User.php</code> — Gestione Profilo	8
4.4.3	<code>api/Category.php</code> — CRUD Categorie	8
4.4.4	<code>api/Item.php</code> — CRUD Oggetti	8
4.4.5	<code>api/Stats.php</code> — Statistiche Avanzate	8
4.4.6	<code>api/Upload.php</code> — Gestione Immagini	9
5	API REST — Endpoint	9
5.1	Autenticazione (<code>/auth</code>)	9
5.2	Utenti (<code>/users</code>)	10
5.3	Categorie (<code>/categories</code>)	10
5.4	Oggetti (<code>/items</code>)	11
5.5	Statistiche (<code>/stats</code>)	12
5.6	Upload Immagini (<code>/upload</code>)	12
6	Frontend (Flutter)	13
6.1	Entry Point (<code>main.dart</code>)	13
6.2	Modelli (<code>models/</code>)	13
6.3	Servizio HTTP (<code>services/api_service.dart</code>)	13
6.4	Provider — State Management	14
6.4.1	<code>AuthProvider</code>	14
6.4.2	<code>CategoryProvider</code>	14
6.4.3	<code>ItemProvider</code>	14
6.4.4	<code>StatsProvider</code>	14

6.5	Navigazione (<code>routing/app_router.dart</code>)	14
6.6	Schermate	15
6.7	Widget Riutilizzabili	15
6.7.1	GlassPanel	15
6.7.2	AnimatedGlassCard	15
6.7.3	AppLayout	15
6.7.4	StarRating	15
7	Flusso Completo — Esempi Pratici	16
7.1	Esempio 1: Login	16
7.2	Esempio 2: Creare un oggetto con immagine	16
7.3	Esempio 3: Statistiche per categoria	16
8	Tecnologie Utilizzate	17
9	Elementi di Innovazione	17
9.1	Autenticazione sicura	17
9.2	Isolamento totale dei dati	17
9.3	Paginazione con metadati	18
9.4	Configurazione separata dal codice	18
9.5	Upload immagini sicuro	18
9.6	Statistiche avanzate	18
9.7	Frontend premium con glassmorphism	18

1 Panoramica Generale

Collectiv è un'applicazione web/mobile pensata per i collezionisti. Permette di organizzare e tenere traccia della propria collezione di oggetti in modo semplice e con un'interfaccia visivamente curata.

Funzionalità principali

- Registrazione e login con autenticazione tramite **token Bearer**
- Creazione di **categorie personalizzate** (es. Fumetti, Monete, Francobolli)
- Aggiunta di oggetti alla collezione con nome, descrizione, **voto da 1 a 5 tramite stelle**, data di acquisizione e immagine
- **Statistiche avanzate** sulla propria collezione
- Gestione del profilo utente

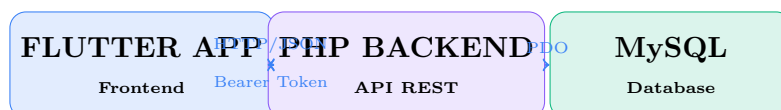
Principio fondamentale: ogni utente vede esclusivamente i propri dati. Categorie e oggetti sono completamente privati e isolati — è strutturalmente impossibile accedere ai dati di un altro utente.

Componenti dell'applicazione

- **Backend in PHP puro** — API RESTful su server Apache
- **Frontend in Flutter** (Dart) — web app con supporto mobile
- **Database MySQL** — per la persistenza dei dati

2 Architettura del Sistema

L'applicazione segue il classico modello **Client-Server** con API REST:



2.1 Flusso di una richiesta tipica

1. L'utente interagisce con l'interfaccia Flutter

2. Il Provider corrispondente chiama il metodo dell'ApiService
3. L'ApiService invia una richiesta HTTP con il token Bearer nell'header
4. Apache riceve la richiesta e, tramite `.htaccess`, la reindirizza a `index.php`
5. `index.php` analizza l'URL e smista al controller PHP corretto
6. Il controller **valida il token**, esegue le query sul database e restituisce JSON
7. Il Provider Flutter riceve i dati, aggiorna lo stato e la UI si ricostruisce

2.2 Struttura delle directory

```
/tpsiproject-main/
  .htaccess           -> Riscrittura URL (Apache)
  config.ini          -> Credenziali database
  database.sql        -> Script creazione database
  index.php           -> Router principale (Front
    Controller)
  |
  api/                -> Controller REST
  |   Auth.php, Category.php, Item.php
  |   Stats.php, Upload.php, User.php
  |
  config/database.php -> Connessione PDO (Singleton)
  core/Auth.php       -> Validazione token Bearer
  core/Response.php   -> Helper risposte JSON
  uploads/items/      -> Immagini caricate
  |
  collectiv/lib/       -> Frontend Flutter
    main.dart, models/, providers/
    routing/, screens/, services/, widgets/
```

Struttura del progetto

3 Database

Il database si chiama **collectiv** e usa MySQL con charset `utf8mb4` e motore **InnoDB** (supporto transazioni e foreign key). È composto da tre tabelle principali.

3.1 Tabella users

Campo	Tipo	Note
id	INT	PK, AUTO_INCREMENT
username	VARCHAR(50)	UNIQUE, NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	Hash bcrypt — la password non viene mai salvata in chiaro
api_token	VARCHAR(128)	Token Bearer, generato al momento del login
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

3.2 Tabella categories

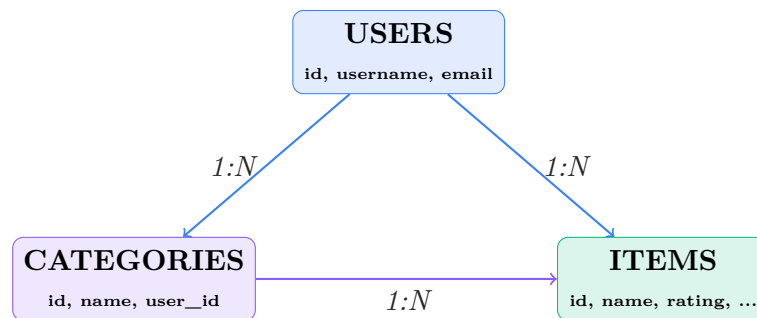
Campo	Tipo	Note
id	INT	PK, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
description	TEXT	Opzionale
user_id	INT	FK → users(id) ON DELETE CASCADE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Vincolo UNIQUE su (name, user_id): un utente non può avere due categorie con lo stesso nome.

3.3 Tabella items

Campo	Tipo	Note
id	INT	PK, AUTO_INCREMENT
name	VARCHAR(150)	NOT NULL
description	TEXT	Opzionale
rating	INT	Valore tra 1 e 10, default 1
acquisition_date	DATE	Opzionale
image_url	VARCHAR(255)	Path relativo dell'immagine
user_id	INT	FK → users(id) ON DELETE CASCADE
category_id	INT	FK → categories(id) ON DELETE CASCADE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

3.4 Schema Relazioni (E/R)



ON DELETE CASCADE: eliminando un utente vengono automaticamente eliminati tutte le sue categorie e tutti i suoi oggetti. Stessa regola vale per l'eliminazione di una categoria.

4 Backend (PHP)

Il backend è scritto in **PHP 8+ puro**, senza framework. Adotta il pattern **Front Controller**: un unico file (`index.php`) riceve tutte le richieste HTTP e le smista ai controller appropriati.

4.1 File di Configurazione

.htaccess — reindirizza ogni richiesta a `index.php`:

```

RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php?url=$1 [QSA,L]
  
```

config.ini — credenziali database, separate dal codice applicativo:

```

[database]
host = "127.0.0.1"
dbname = "collectiv"
user = "root"
password = ""
  
```

config/database.php — connessione PDO con pattern **Singleton**: la connessione viene creata una sola volta e riutilizzata per tutta la durata della richiesta. Legge i parametri da `config.ini` e configura PDO per usare prepared statements nativi e restituire array associativi.

4.2 Router Principale (index.php)

Analizza l'URL ricevuto ed esegue il routing:

Risorsa URL	Controller
/auth/...	api/Auth.php
/items/...	api/Item.php
/users/...	api/User.php
/stats/...	api/Stats.php
/upload/...	api/Upload.php
/categories/...	api/Category.php

Esempio: GET /items/5 → \$resource = "items", \$param1 = "5" → chiama `handleItems($pdo, "GET", "5")`

4.3 Moduli di Supporto (core/)

4.3.1 core/Auth.php

Funzione `requireUserAuth($pdo)`: legge l'header `Authorization: Bearer <token>`, cerca il token nel database e restituisce i dati dell'utente. Se il token non è valido restituisce errore 401.

Viene chiamata all'inizio di **ogni controller** (eccetto login e registrazione), garantendo che ogni operazione sia associata all'utente corretto.

4.3.2 core/Response.php

Helper per standardizzare tutte le risposte JSON dell'API:

- `sendResponse($statusCode, $data)` — invia una risposta JSON con il codice HTTP specificato
- `sendError($statusCode, $message)` — invia un errore in formato JSON standardizzato (ispirato a RFC 7807)
- `getJsonInput()` — legge e decodifica il body JSON delle richieste POST/PUT

4.4 Controller API

4.4.1 api/Auth.php — Autenticazione

- **Registrazione:** verifica i campi, genera l'hash bcrypt della password con `password_hash(PASSWORD_BCRYPT)`, inserisce l'utente. Il token non viene generato alla registrazione; l'utente deve fare login separatamente.

- **Login:** verifica email e password con `password_verify()`, genera un token sicuro con `bin2hex(random_bytes(32))` (64 caratteri esadecimali), lo salva nel database e lo restituisce al client.
- **/auth/me:** verifica il token e restituisce i dati dell'utente loggato. Usato dal frontend per verificare che il token salvato sia ancora valido.

4.4.2 api/User.php — Gestione Profilo

Permette di leggere, aggiornare ed eliminare l'account. Un utente può modificare o eliminare **solo il proprio profilo**, non quelli altrui (errore 403 in caso contrario).

4.4.3 api/Category.php — CRUD Categorie

Ogni query include sempre `WHERE user_id = :uid`, quindi un utente vede e modifica solo le proprie categorie. L'eliminazione di una categoria cancella a cascata tutti gli oggetti associati.

4.4.4 api/Item.php — CRUD Oggetti

Il controller più complesso. Caratteristiche principali:

- La lista oggetti supporta **paginazione** (`LIMIT/OFFSET`) e **ricerca** per nome (`LIKE`)
- La risposta include metadati (`total_items`, `total_pages`) per gestire la paginazione lato client
- Alla creazione, viene verificato che la `category_id` specificata appartenga all'utente loggato

4.4.5 api/Stats.php — Statistiche Avanzate

Esegue multiple query SQL aggregate e restituisce un JSON strutturato con:

1. **Overview:** totale oggetti, media voti, deviazione standard, oggetti con voto 10, oggetti con immagine, oggetti degli ultimi 30 giorni, **Collector Score**
2. **Distribuzione per categoria:** numero di oggetti per ogni categoria
3. **Distribuzione voti:** quanti oggetti per ogni voto da 1 a 5
4. **Timeline acquisizioni:** oggetti acquisiti per anno
5. **Record personali:** oggetto con voto più alto, acquisizione più vecchia, più recente, categoria migliore

Il **Collector Score** è un punteggio personalizzato:

$$Score = (totaleoggetti \times mediavoti) + (oggettivoto10 \times 50) + (oggetticonimmagine \times 10)$$

4.4.6 api/Upload.php — Gestione Immagini

Flusso di sicurezza per ogni upload:

1. Verifica autenticazione e che l'oggetto appartenga all'utente
2. Controlla il **MIME type reale** del file (non solo l'estensione), tramite `finfo_open(FILEINFO_MIME_TYPE)`
3. Verifica che la dimensione non superi 5MB
4. Genera un nome univoco con `uniqid()` per evitare collisioni
5. Salva il file in `/uploads/items/` ed elimina automaticamente la vecchia immagine
6. Aggiorna il campo `image_url` nel database

5 API REST — Endpoint

Base URL: `http://localhost/web/`

Tutte le richieste (eccetto `/auth/register` e `/auth/login`) richiedono l'header:
`Authorization: Bearer <token>`

5.1 Autenticazione (/auth)

Metodo	Endpoint	Descrizione
POST	<code>/auth/register</code>	Registra un nuovo utente
POST	<code>/auth/login</code>	Login, restituisce il token
GET	<code>/auth/me</code>	Dati dell'utente loggato

```
1 // Request Body
2 { "email": "mario@test.it", "password": "password123" }
3
4 // Response 200 OK
5 {
6   "message": "Login completato con successo.",
7   "token": "9a8926f6d5e74289b779ed642e9cc4ce...",
8   "user": { "id": 2, "username": "mario", "email": "mario@test.it" }
9 }
```

POST `/auth/login` — Request & Response

5.2 Utenti (/users)

Metodo	Endpoint	Descrizione
GET	/users	Lista utenti (solo id e username)
GET	/users/{id}	Dettaglio di un utente
PUT	/users/{id}	Aggiorna il proprio profilo
DELETE	/users/{id}	Elimina il proprio account

Un utente può modificare o eliminare **solo il proprio account**. Tentare di accedere all'account altrui restituisce errore **403 Forbidden**.

5.3 Categorie (/categories)

Metodo	Endpoint	Descrizione
GET	/categories	Lista delle proprie categorie
GET	/categories/{id}	Dettaglio categoria
POST	/categories	Crea una nuova categoria
PUT	/categories/{id}	Aggiorna una categoria
DELETE	/categories/{id}	Elimina categoria e tutti i suoi oggetti

```
1 // Request Body
2 { "name": "Fumetti Rari", "description": "I fumetti piu' rari
   della mia collezione" }
3
4 // Response 201 Created
5 { "message": "Categoria creata con successo.", "id": 14 }
```

POST /categories — Request & Response

Attenzione: l'eliminazione di una categoria è irreversibile e cancella automaticamente tutti gli oggetti associati (ON DELETE CASCADE).

5.4 Oggetti (/items)

Metodo	Endpoint	Descrizione
GET	/items	Lista paginata degli oggetti
GET	/items/{id}	Dettaglio completo di un oggetto
POST	/items	Crea un nuovo oggetto
PUT	/items/{id}	Aggiorna un oggetto
DELETE	/items/{id}	Elimina un oggetto

La lista supporta parametri opzionali: `?page=1&limit=20&search=spider`

```

1 {
2   "data": [
3     {
4       "id": 1,
5       "name": "spider",
6       "rating": 5,
7       "image_url": "/uploads/items/item_1_abc123.png",
8       "category_id": 1
9     }
10  ],
11  "meta": {
12    "current_page": 1,
13    "per_page": 20,
14    "total_items": 3,
15    "total_pages": 1
16  }
17 }
```

GET /items — Response con paginazione

```

1 // Request Body ("name" e "category_id" sono obbligatori)
2 {
3   "name": "Nuovo fumetto",
4   "description": "Un fumetto molto raro",
5   "rating": 8,
6   "acquisition_date": "2024-03-15",
7   "category_id": 1
8 }
9 // Response 201 Created
10 { "message": "Oggetto creato con successo nella tua collezione", "id": 44 }
```

POST /items — Creare un oggetto

5.5 Statistiche (/stats)

Metodo	Endpoint	Descrizione
GET	/stats	Statistiche globali
GET	/stats/{category_id}	Statistiche filtrate per categoria

```

1 {
2   "overview": {
3     "scope": "All Collection",
4     "collector_score": 265,
5     "total_items": 3,
6     "average_rating": 4.67,
7     "perfect_items": 0,
8     "items_with_images": 3,
9     "total_categories": 1
10  },
11  "by_category": [
12    { "id": 1, "category": "spider-man", "count": 3, "
13      avg_rating": 4.67 }
14  ],
15  "rating_distribution": [
16    { "rating": 5, "count": 2 }, { "rating": 4, "count": 1 }
17  ],
18  "acquisition_timeline": [
19    { "year": 2020, "count": 1 }, { "year": 2025, "count": 1 }
20  ],
21  "records": {
22    "topRated": { "id": 1, "name": "spider", "rating": 5 },
23    "oldest_acquisition": { "id": 3, "name": "spider", "
24      acquisition_date": "2020-05-14" },
25    "best_category": { "name": "spider-man", "avg_rating": 4.67
26  }
27 }

```

GET /stats — Response (esempio)

5.6 Upload Immagini (/upload)

Metodo	Endpoint	Descrizione
POST	/upload/{item_id}	Carica un'immagine per un oggetto
DELETE	/upload/{item_id}	Elimina l'immagine di un oggetto

Vincoli: formati accettati JPEG, PNG, WEBP; dimensione massima 5 MB; invio come multipart/form-data con campo "image".

6 Frontend (Flutter)

Il frontend è un'app Flutter che gira come **web app** (compatibile anche con Android/iOS). Usa il pattern **Provider** per la gestione dello stato applicativo.

6.1 Entry Point (`main.dart`)

All'avvio, l'app verifica se esiste un token salvato localmente. Se sì, lo valida con `GET /auth/me`; se è scaduto, effettua il logout automatico. L'app usa **Material Design 3** con tema dark e font moderni.

Colore	Hex
Primary (Blu)	#3B82F6
Secondary (Viola)	#8B5CF6
Accent (Verde)	#10B981
Error (Rosso)	#EF4444
Surface (Sfondo)	#09090B

6.2 Modelli (`models/`)

Classi Dart che rappresentano le entità del database. Ogni modello ha:

- Un costruttore `fromJson()` per creare l'oggetto dalla risposta API
- Un metodo `toJson()` per serializzarlo nelle richieste

Modelli presenti: `User`, `Category`, `Item`, e la struttura complessa `FullStats` con le sottoclassi `StatsOverview`, `CategoryStat`, `RatingBucket`, `TimelineEntry`, `StatsRecords`.

La classe `Item` aggiunge automaticamente un parametro `?v=timestamp` all'URL dell'immagine per evitare problemi di cache del browser.

6.3 Servizio HTTP (`services/api_service.dart`)

Classe `ApiService`: gestisce tutta la comunicazione con il backend. Legge il token da `SharedPreferences` e lo aggiunge automaticamente all'header di ogni richiesta.

Metodi principali:

Metodo	Descrizione
<code>get(endpoint, queryParams)</code>	Richiesta HTTP GET con parametri opzionali
<code>post(endpoint, body)</code>	Richiesta HTTP POST con body JSON
<code>put(endpoint, body)</code>	Richiesta HTTP PUT con body JSON
<code>delete(endpoint)</code>	Richiesta HTTP DELETE
<code>uploadMultipart(...)</code>	Upload file come <code>multipart/form-data</code>

In caso di errore HTTP, lancia un'eccezione `ApiException` con codice e messaggio.

6.4 Provider — State Management

Classi che estendono `ChangeNotifier`. I widget ascoltano i provider e si aggiornano automaticamente quando i dati cambiano.

6.4.1 AuthProvider

Gestisce login, registrazione, profilo e logout. Alla disconnessione resetta tutto lo stato e il router reindirizza automaticamente alla pagina di login.

6.4.2 CategoryProvider

Mantiene la lista delle categorie in memoria e offre metodi per crearle, modificarle ed eliminarle, ricaricando la lista dopo ogni operazione.

6.4.3 ItemProvider

Gestisce la lista oggetti con **paginazione infinita** e filtro per categoria. Ogni caricamento aggiunge i nuovi elementi a quelli già presenti. Tiene traccia di `_hasMore` per sapere se ci sono altre pagine disponibili.

6.4.4 StatsProvider

Scarica e mantiene le statistiche globali. Supporta anche il fetch filtrato per categoria senza aggiornare lo stato interno (usato per visualizzazioni temporanee).

6.5 Navigazione (`routing/app_router.dart`)

Usa **GoRouter** con navigazione dichiarativa e redirect automatici:

- Se non autenticato → redirect a `/login`
- Se autenticato e su `/login` o `/register` → redirect a `/`

Tutte le pagine protette sono avvolte da `AppLayout` tramite `ShellRoute`, che fornisce la sidebar di navigazione.

6.6 Schermate

Schermata	Descrizione
LoginScreen	Form di login con design glassmorphism
RegisterScreen	Registrazione con validazione e conferma password
HomeScreen	Dashboard con Collector Score, metriche, record e oggetti recenti
CategoriesScreen	CRUD categorie: swipe per eliminare, dialog per creare/modificare
ItemsScreen	Lista/griglia oggetti con ricerca, filtro per categoria, scroll infinito e upload immagini
StatsScreen	Grafici interattivi (distribuzione voti, timeline), filtrabile per categoria
SettingsScreen	Modifica profilo, cambio password, logout, eliminazione account

6.7 Widget Riutilizzabili

6.7.1 GlassPanel

Implementa l'effetto **glassmorphism** (vetro smerigliato) tramite `BackdropFilter` con blur. Usato come base per tutte le card dell'app. Parametri configurabili: `borderRadius`, `blur`, `opacity`, `borderColor`.

6.7.2 AnimatedGlassCard

Versione animata del `GlassPanel`. Al passaggio del mouse o al tocco:

- L'elemento si ingrandisce leggermente (scala da 1.0 a 1.025)
- Appare un glow colorato sul bordo
- Animazione da 200ms con curva `easeOutCubic`

6.7.3 AppLayout

Layout principale con sidebar di navigazione. Su schermi larghi la sidebar è fissa; su schermi stretti diventa un drawer a comparsa (hamburger menu).

6.7.4 StarRating

Widget per visualizzare e selezionare un voto con stelle (1–5). Disponibile in modalità **sola lettura** o **interattiva**. Stelle piene in ambra (`#F59E0B`) per i voti assegnati, stelle vuote per i rimanenti.

7 Flusso Completo — Esempi Pratici

7.1 Esempio 1: Login

1. L'app si avvia → nessun token salvato → GoRouter reindirizza a `/login`
2. L'utente inserisce email e password e preme "Accedi"
3. `AuthProvider` chiama `POST /auth/login` tramite `ApiService`
4. Il backend verifica le credenziali con `password_verify()`, genera il token e lo restituisce
5. Il token viene salvato in `SharedPreferences`
6. `AuthProvider` imposta `_isAuthenticated = true` e chiama `notifyListeners()`
7. GoRouter rileva il cambio di stato e reindirizza alla home (`/`)

7.2 Esempio 2: Creare un oggetto con immagine

1. L'utente preme il pulsante `+` su `ItemsScreen`
2. Compila il form (nome, voto, categoria...) e conferma
3. `ItemProvider` chiama `POST /items` — il backend verifica che la categoria appartenga all'utente e inserisce l'oggetto
4. L'utente sceglie un'immagine dalla galleria tramite `image_picker`
5. `ItemProvider` chiama `uploadMultipart('/upload/{id}', ...)`
6. Il backend valida il MIME type, salva il file, aggiorna `image_url` nel database
7. La lista si ricarica e l'immagine appare sull'oggetto

7.3 Esempio 3: Statistiche per categoria

1. L'utente apre `/stats` → vengono caricate le statistiche globali con `GET /stats`
2. Il backend esegue 5+ query aggregate con funzioni `COUNT`, `AVG`, `STDDEV`
3. L'utente seleziona una categoria dal dropdown
4. `StatsProvider` chiama `GET /stats/{category_id}`
5. Il backend riesegue le stesse query aggiungendo il filtro `AND category_id = :cid`
6. I grafici si aggiornano con i dati filtrati

8 Tecnologie Utilizzate

Backend	
PHP 8+	Senza framework — logica applicativa in PHP puro
Apache + mod_rewrite	Server web con riscrittura URL tramite <code>.htaccess</code>
MySQL / MariaDB	Database relazionale con InnoDB
PDO + Prepared Statements	Connessione sicura al database, protezione da SQL Injection
<code>password_hash()</code> <code>bcrypt</code>	Hashing sicuro delle password
Frontend	
Flutter 3.8+ / Dart	Framework cross-platform (web, Android, iOS)
Provider	State management con <code>ChangeNotifier</code>
GoRouter	Navigazione dichiarativa con redirect automatici
<code>http</code> + <code>shared_preferences</code>	Chiamate HTTP e persistenza locale del token
<code>fl_chart</code>	Grafici interattivi (barre, linee)
Material Design 3	Design system con tema dark personalizzato
Comunicazione	
API REST	Architettura stateless con risorse JSON
Bearer Token	Autenticazione senza sessioni server-side
HTTP Methods	GET, POST, PUT, DELETE

9 Elementi di Innovazione

9.1 Autenticazione sicura

Password hashate con **bcrypt** — un algoritmo lento per design, che rende impraticabile il brute force. Il token di sessione viene generato con `random_bytes(32)`, una funzione crittograficamente sicura, producendo 64 caratteri esadecimali. Il token non viene mai riutilizzato tra sessioni diverse.

9.2 Isolamento totale dei dati

Ogni query SQL include sempre la clausola `WHERE user_id = :uid`. Questo non è solo un controllo logico ma una garanzia strutturale: è **impossibile** accedere ai dati di un altro utente, anche conoscendone gli ID delle risorse. Le operazioni di scrittura verificano l'appartenenza prima di procedere.

9.3 Paginazione con metadati

L'endpoint `/items` restituisce sia i dati che le informazioni di paginazione (`total_pages`, `total_items`, `has_more`), permettendo al frontend di implementare lo **scroll infinito** senza conoscere in anticipo la dimensione della collezione.

9.4 Configurazione separata dal codice

Le credenziali del database sono in un file `config.ini` esterno, secondo il principio di separazione tra configurazione e logica applicativa. Questo rende più sicuro il deploy e più semplice la gestione di ambienti diversi (sviluppo, produzione).

9.5 Upload immagini sicuro

La validazione del file avviene sul **MIME type reale** (tramite `finfo_open`), non sulla sola estensione del nome file — un controllo facilmente aggirabile. I nomi file vengono generati con `uniqid()` per evitare collisioni e la vecchia immagine viene eliminata automaticamente in caso di sostituzione.

9.6 Statistiche avanzate

Il **Collector Score** è una metrica originale che misura la "potenza" di una collezione premiando quantità, qualità e completezza. Le statistiche includono distribuzione voti, timeline per anno e record personali, tutte filtrabili per categoria tramite un unico endpoint.

9.7 Frontend premium con glassmorphism

Il design adotta l'effetto **glassmorphism** (vetro smerigliato) con `BackdropFilter` e blur. Le micro-animazioni al hover/touch (scala + glow) danno un feedback visivo immediato. Il tema dark con palette armoniosa e i grafici interattivi con `fl_chart` rendono l'esperienza utente moderna e piacevole.

COLLECTIV

Progetto di TPSIT — Classe 5CII

Girolimetto Riccardo · Muca Alesander · Touel Adem

Anno Scolastico 2025/2026